Table of Contents

I. INTRODUCTION 3
II. C PROGRAM STRUCTURE AND EXAMPLES 3
A. Basic Hello World Program 3
Summary 3
B. Program with User Input 3
Summary 3
C. Program with Formatted Output 4
Summary 4
III. GCC COMPILATION PROCESS 4
A. Installing GCC on Linux 4
Summary 4
B. Four Stages of Compilation 5
Stage 1: Preprocessing 5
Stage 2: Compilation to Assembly 5
Stage 3: Assembly to Object File 6
Stage 4: Linking 6
IV. RETURN VALUES AND EXIT STATUS 6
A. Understanding Return Values 6
Summary 6
B. Checking Exit Status 7
V. KNOWLEDGE CHECK QUESTIONS 7
A. Question 1 7
Q: What is the purpose of the #include directive in C programming? Provide an example of its usage. 7
B. Question 2 7
Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage? 7
C. Question 3 8
Q: What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value? 8
D. Question 4 8
Q: Write a C program that: • Declares an integer variable • Asks the user to input a value • Displays the entered value back to the user 8
E. Question 5 8
Q: What GCC command would you use to: 8
1) Only preprocess a file without compiling? 8
2) Generate assembly code but not assemble it? 8
3) Create an object file without linking? 8
F. Question 6 9
Q: Explain why C is called a ”compiled language” and how this differs from interpreted languages. 9
VIII. CONCLUSION 9

Table of Figures

Figure 1: Basic Hello World Program 3
Figure 2: Program with user input 3
Figure 3: Program with Formatted Output 4
Figure 4: GCC Installation Commands 4
Figure 5: Four Stages of Compilation 5
Figure 6: Preprocessing Stage 5
Figure 7: Compilation Stage 5
Figure 8: Assembly Stage 6
Figure 9: Linking Stage 6
Figure 10: Program with Return Value 6
Figure 11: Checking Program Exit Status 7

I. INTRODUCTION

This report documents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The focus is on understanding C program structure, compilation stages, and basic Linux commands for software development.

II. C PROGRAM STRUCTURE AND EXAMPLES

A. Basic Hello World Program



Figure 1: Basic Hello World Program

Summary

The above code demonstrates a basic C program structure with preprocessor directives, main function, and return statement.

B. Program with User Input

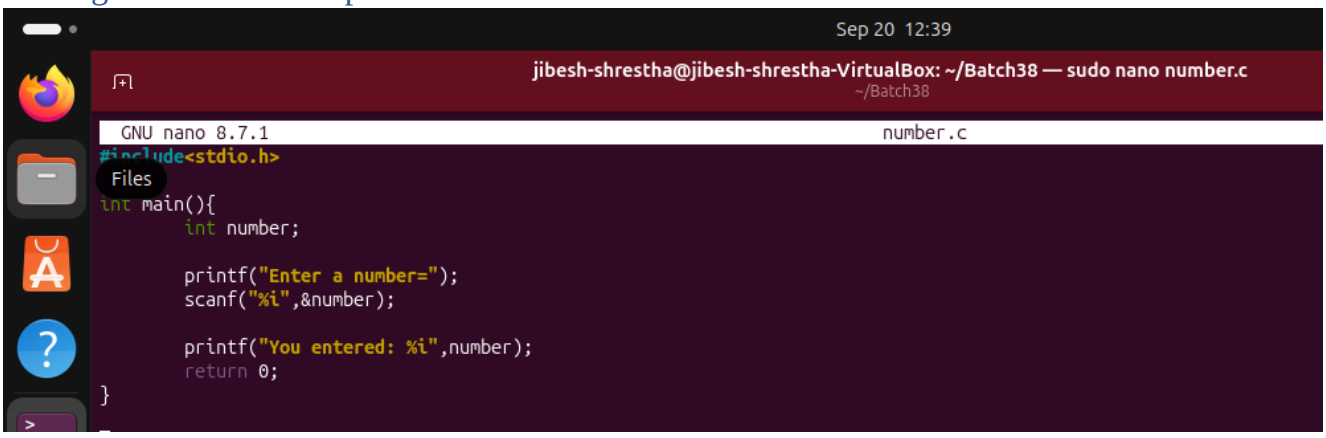


Figure 2: Program with user input

Summary

This code demonstrates the use of printf and scanf for input/output operations in C language.

C. Program with Formatted Output



Figure 3: Program with Formatted Output

Summary

This code shows multiple format specifiers and variables.

III. GCC COMPILATION PROCESS

A. Installing GCC on Linux

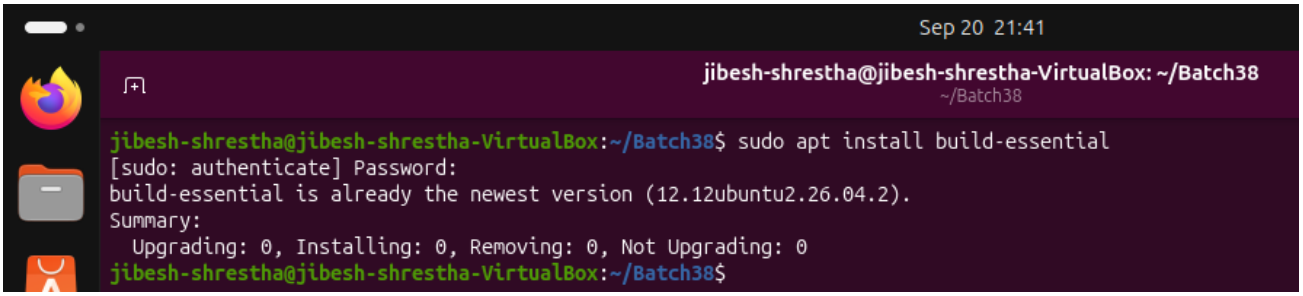


Figure 4: GCC Installation Commands

Summary

The above commands are used to install and verify GCC

B. Four Stages of Compilation

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

THE FOUR STAGES OF C COMPILATION

Stage	Command	Output	Purpose
1. Preprocessing	`gcc -E hello.c -o hello.i`	`.i` file	Expand headers & macros
2. Compiling	`gcc -S hello.i -o hello.s`	`.s` file	Generate Assembly code
3. Assembling	`gcc -c hello.s -o hello.o`	`.o` file	Create object file
4. Linking	`gcc hello.o -o hello`	executable	Final executable

Figure 5: Four Stages of Compilation

Stage 1: Preprocessing

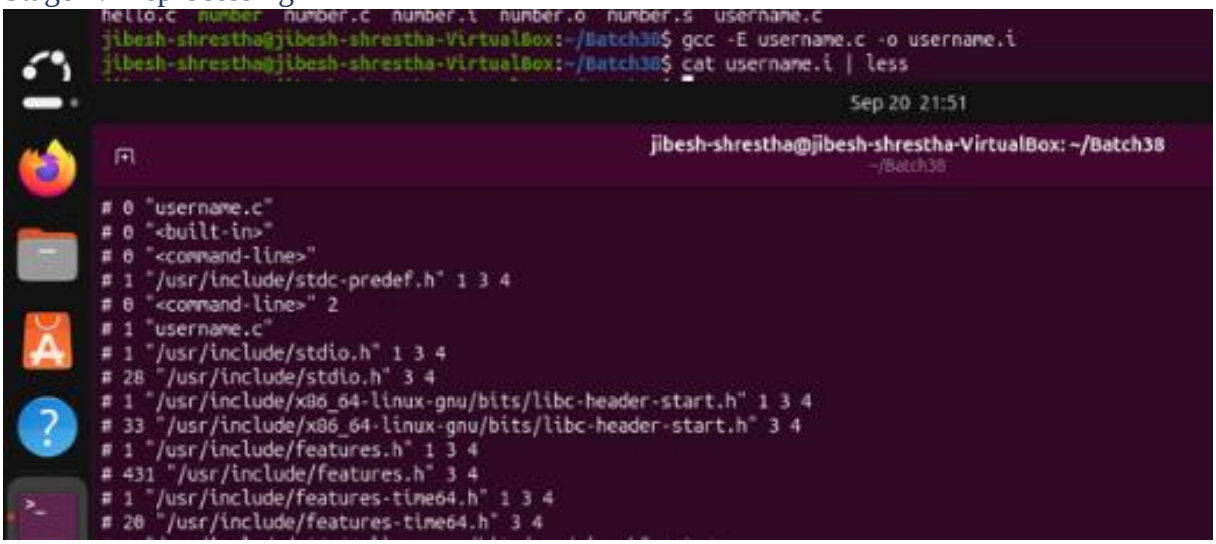


Figure 6: Preprocessing Stage

Stage 2: Compilation to Assembly

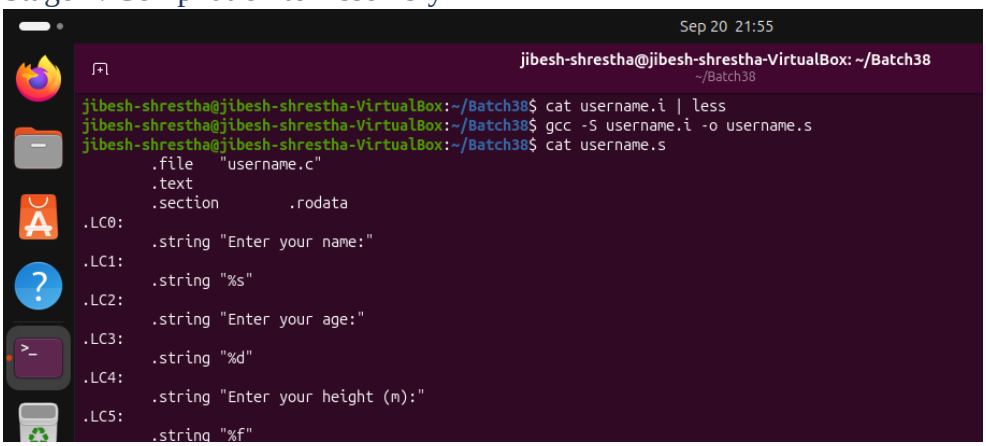


Figure 7: Compilation Stage

Stage 3: Assembly to Object File

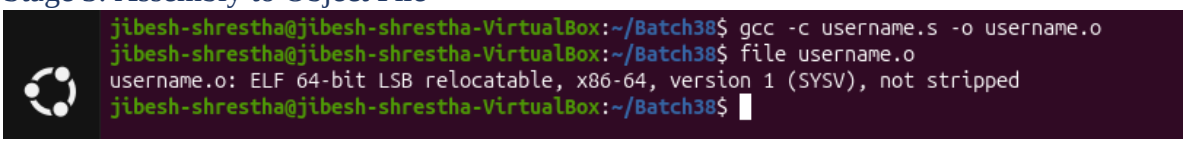


Figure 8: Assembly Stage

Stage 4: Linking

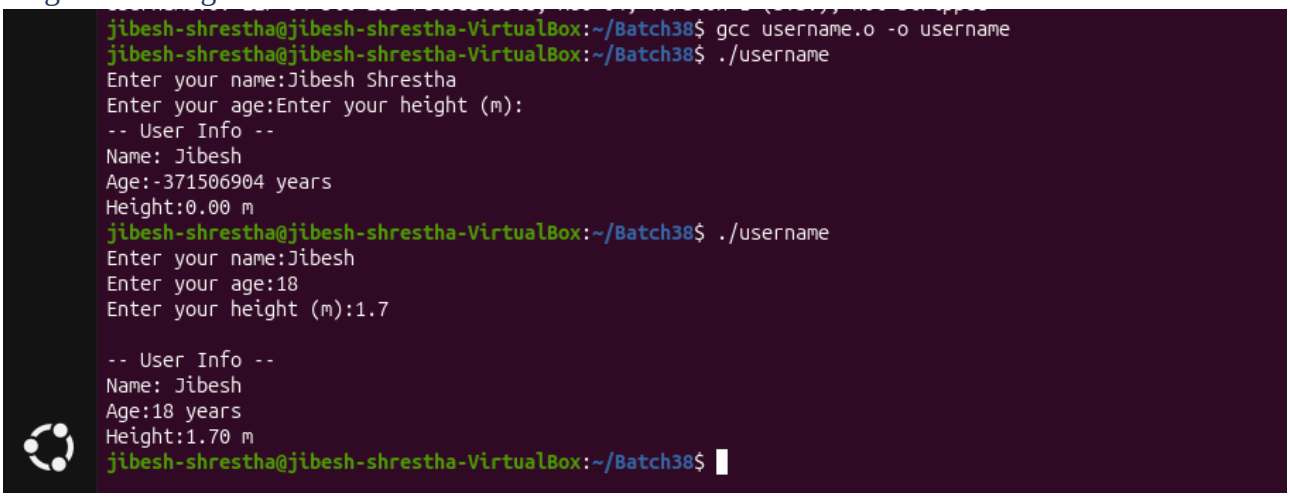


Figure 9: Linking Stage

IV. RETURN VALUES AND EXIT STATUS

A. Understanding Return Values

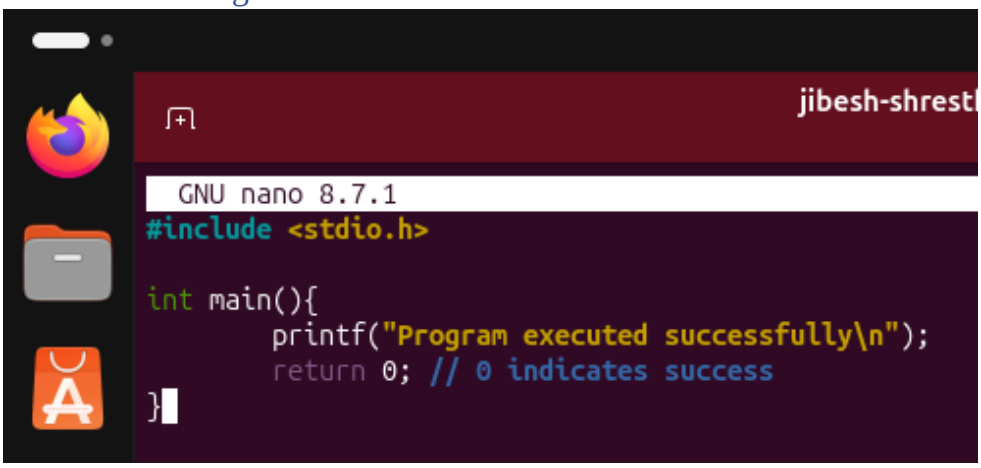


Figure 10: Program with Return Value

Summary

The return value of main() indicates program success or failure to the OS

B. Checking Exit Status

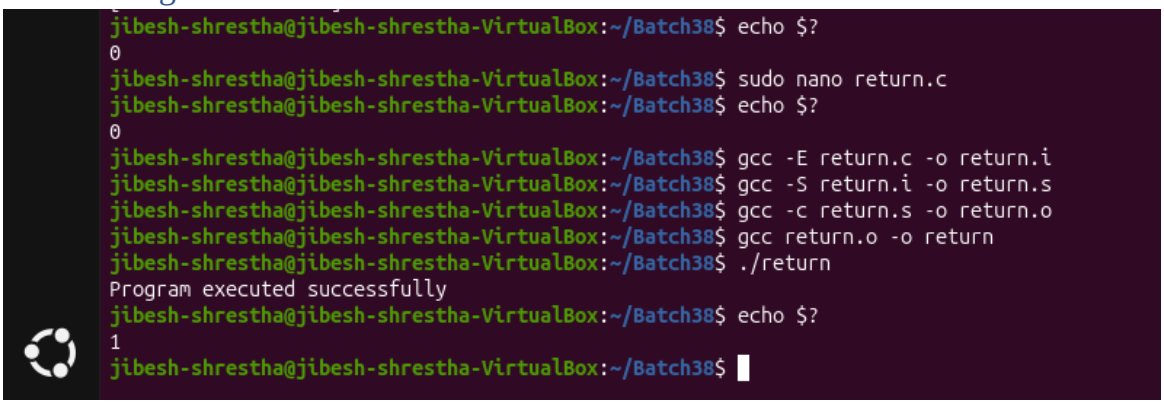


Figure 11: Checking Program Exit Status

V. KNOWLEDGE CHECK QUESTIONS

A. Question 1

Q: What is the purpose of the #include directive in C programming? Provide an example of its usage.

Ans: #include is a preprocessor directive that tells the compiler to insert the contents of another file into your source file before compilation actually begins.

B. Question 2

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Ans:

1. Preprocessing (.c -> .i): Cleans up the code by removing comments and expanding shortcuts (like copying in #include files).

2. Compilation (.i -> .s): Translates the clean C code into Assembly language, which is a low-level text language meant for your specific computer chip.

3. Assembly (.s -> .o or .obj): Converts that Assembly text into binary Object code (raw 1s and Os) that the computer hardware can read.

4. Linking (.o -> .exe or .out): Glues your object code together with standard library functions (like printf) to create the final, runnable Executable program.

C. Question 3

Q: What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

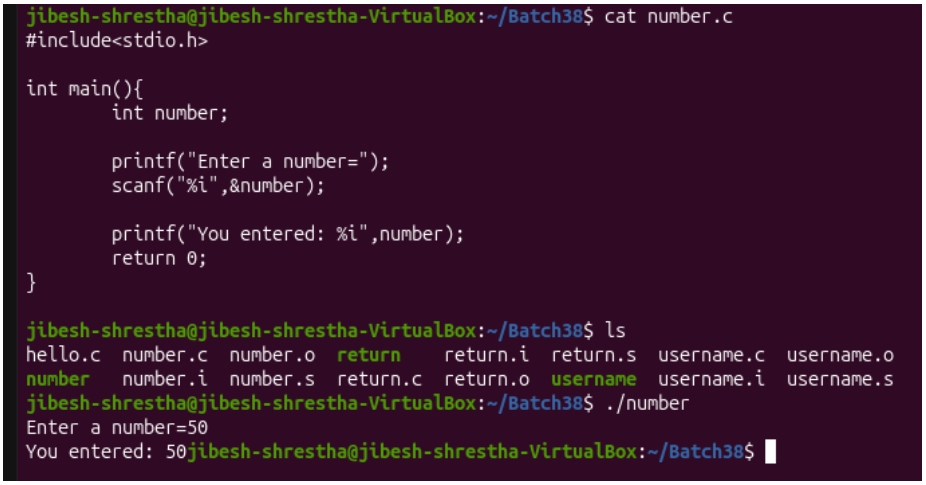
Ans:

A return value of 0 from the main() function indicates to the operating system that the program executed successfully and closed without any errors.A non-zero return value (typically 1 or higher) indicates that the program encountered an error or closed abnormally.

D. Question 4

Q: Write a C program that: • Declares an integer variable • Asks the user to input a value • Displays the entered value back to the user

Ans:



```
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ cat number.c
#include<stdio.h>

int main(){
        int number;

        printf("Enter a number=");
        scanf("%i",&number);

        printf("You entered: %i",number);
        return 0;
}

jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ ls
hello.c  number.c  number.o  return    return.i  return.s  username.c  username.o
number   number.i  number.s  return.c  return.o  username  username.i  username.s
jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$ ./number
Enter a number=50
You entered: 50jibesh-shrestha@jibesh-shrestha-VirtualBox:~/Batch38$
```

E. Question 5

Q: What GCC command would you use to:

1) Only preprocess a file without compiling?

2) Generate assembly code but not assemble it?

3) Create an object file without linking?

Ans:

1) gcc -E main.c -o main.i (Use -E)

2) gcc -S main.c -o main.s (Use -S)

3) gcc -c main.c -o main.o (Use -c)

F. Question 6

Q: Explain why C is called a ”compiled language” and how this differs from interpreted languages.
Ans: C is called a compiled language because its entire source code must be translated into binary machine code (via the four stages of compilation) ahead of time before the CPU can execute it. This makes compiled programs **significantly faster** than interpreted ones, as the computer runs the ready-to-go machine code directly, **without needing to translate it line-by-line** while the program is running.

VIII. CONCLUSION

This lab exercise provided hands-on experience with C programming fundamentals, including program structure, compilation process, and GCC usage. Understanding these concepts is essential for systems programming and operating system development. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions.